Tarefa 12: Avaliação de Escalabilidade do Código de Navier-Stokes (OpenMP)

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

O objetivo desta tarefa é avaliar a escalabilidade do código de Navier-Stokes 2D (apenas viscosidade, esquema explícito de 5 pontos, contorno periódico, malha 1024×1024) em um nó de computação do NPAD, identificar os gargalos e mostrar como cada versão do código mudou a escalabilidade da anterior.

Conceitos usados.

- **Desempenho** é a rapidez da execução ($1/T$). **Escalabilidade** é a eficiência com que usamos os recursos. Uma versão pode ser mais rápida e, ainda assim, escalar pior.
- **Speedup:** $S(p) = T(1)/T(p)$. **Eficiência:** $E(p) = S(p)/p$. Eficiência 100% significa que os p cores estão sendo totalmente aproveitados.
- **Escalabilidade forte:** o problema tem tamanho fixo e aumentamos os cores. O ideal é manter E constante, mas a parte serial e os custos que não diminuem com p (sincronização, memória) limitam isso (modelo de Amdahl).
- **Escalabilidade fraca:** o problema cresce junto com os cores (trabalho por core constante). O ideal também é manter E constante (modelo de Gustafson).
- **Programa escalável:** é possível compensar a perda de eficiência causada por mais cores aumentando o tamanho do problema.

2 Metodologia

Ferramentas. OpenMP para a paralelização, supercomputador NPAD (um nó da partição `amd-512`, usado de forma exclusiva) e `pascalanalyzer` com instrumentação manual, cujos relatórios foram visualizados no PaScal Viewer.

Versões avaliadas. A Tabela 1 resume o que muda em cada versão. Os trechos principais do código estão no Apêndice.

Versão	Mudança principal	Schedule	Região paralela
1	Paraleliza só o laço externo `i`	`runtime`	a cada passo
2	`collapse(2)` nos laços `i` e `j`	`static`	a cada passo
3	Laço interno sem condicionais + SIMD	`static`	a cada passo
4	Uma única região paralela para todos os passos	`static`	única

Tabela 1: Versões do código. Na versão 1, o schedule foi variado por `OMP_SCHEDULE`: `static`, `dynamic,1`, `dynamic,16` e `guided`.

Execução. Cores: 1, 2, 4, 8, 16, 32 e 64. Seis entradas (i1 a i6), com t_{final} de 0,005 a 0,16, dobrando a cada entrada, o que dobra o trabalho (número de passos), com $N = 1024$ fixo. Três repetições por medição e threads fixadas aos cores (`OMP_PROC_BIND=close`, `OMP_PLACES=cores`). A região medida é o passo de tempo; na versão 4, ela cobre todos os passos.

Como ler os heatmaps. Cada linha é um número de cores (2 a 64) e cada coluna é uma entrada (i1 a i6). No mapa de *Efficiency*, escuro é eficiência alta (próxima de 100%) e claro é eficiência baixa. Nos demais, azul é positivo e vermelho é negativo. Interpretamos a *Strong Scalability* como a evolução descendo uma coluna (mesma entrada, mais cores) e a *Weak Scalability* como avançar uma coluna dobrando os cores (trabalho por core constante).

3 Resultados

As Figuras 1 a 7 mostram os heatmaps de cada configuração e a Tabela 2 resume a leitura deles.

Versão	Eficiência com 2–8 cores	Onde a eficiência cai	Strong / Weak
1, `dynamic,1`	Já abaixo das demais	A partir de 16 cores	Vermelho / vermelho
1, `dynamic,16` e `guided`	Alta (2–4 cores)	A partir de 16 cores	Vermelho / vermelho
1, `static`	Alta até 8–16 cores	Em 32–64 cores	Vermelho mais claro
2, `collapse`	Como a 1 `static`	Em 32–64 cores, um pouco depois	Vermelho mais claro
3, SIMD	Alta	Em 32–64 cores, parecida ou um pouco pior que a 2	Vermelho mais claro
4, região única	Alta	Em 64 cores; melhor nas entradas maiores	Vermelho mais claro

Tabela 2: Resumo qualitativo dos heatmaps.

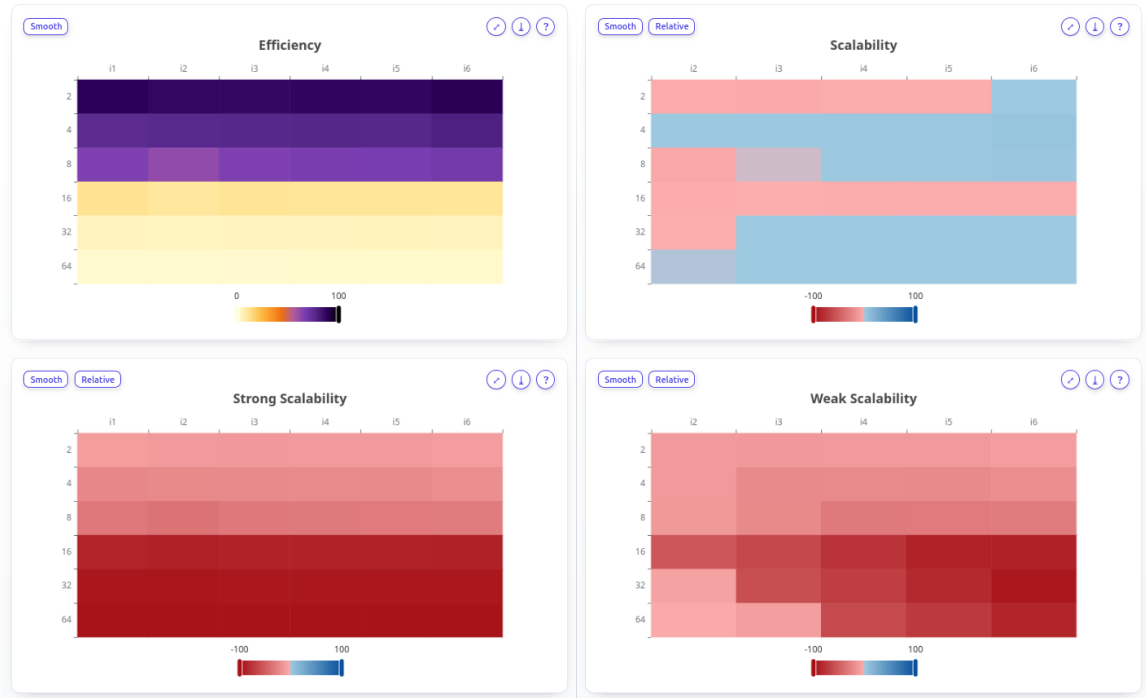


Figura 1: Versão 1, `schedule(dynamic,1)`.

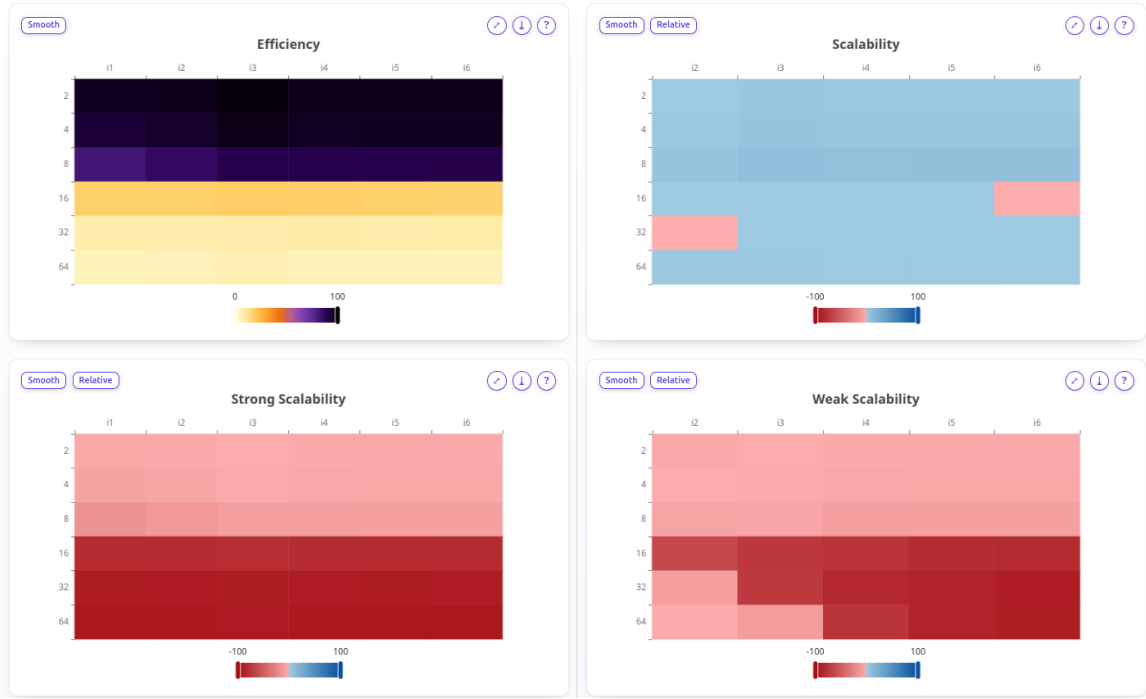


Figura 2: Versão 1, `schedule(dynamic,16)`.

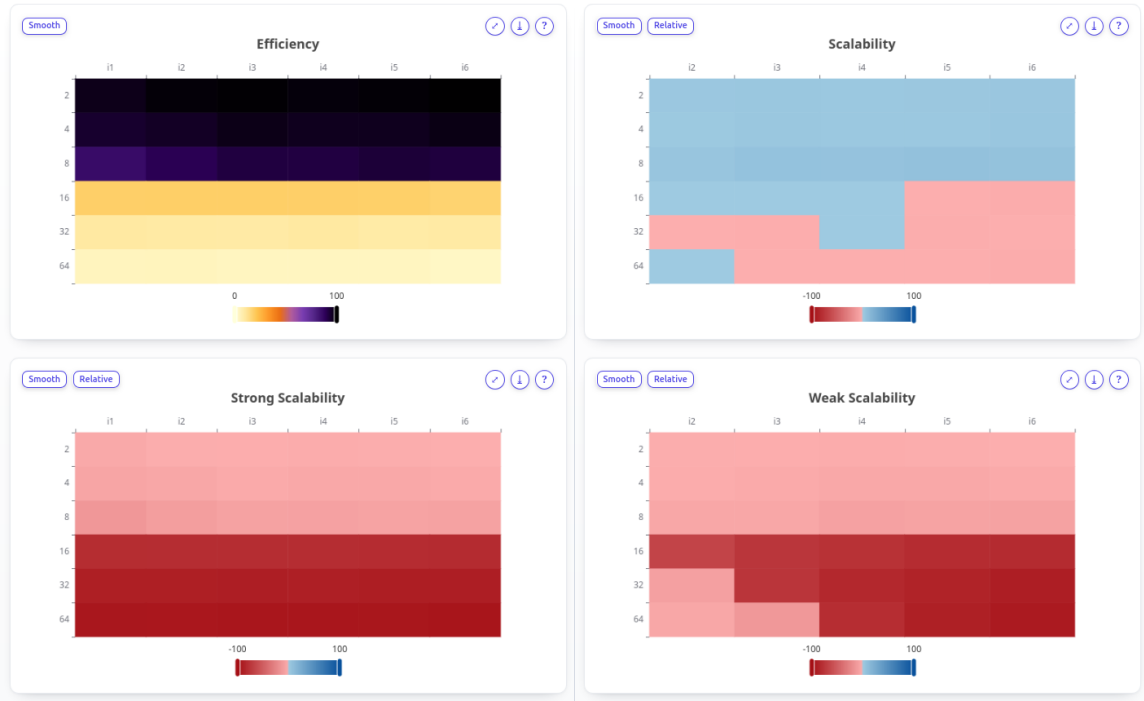


Figura 3: Versão 1, `schedule(guided)`.

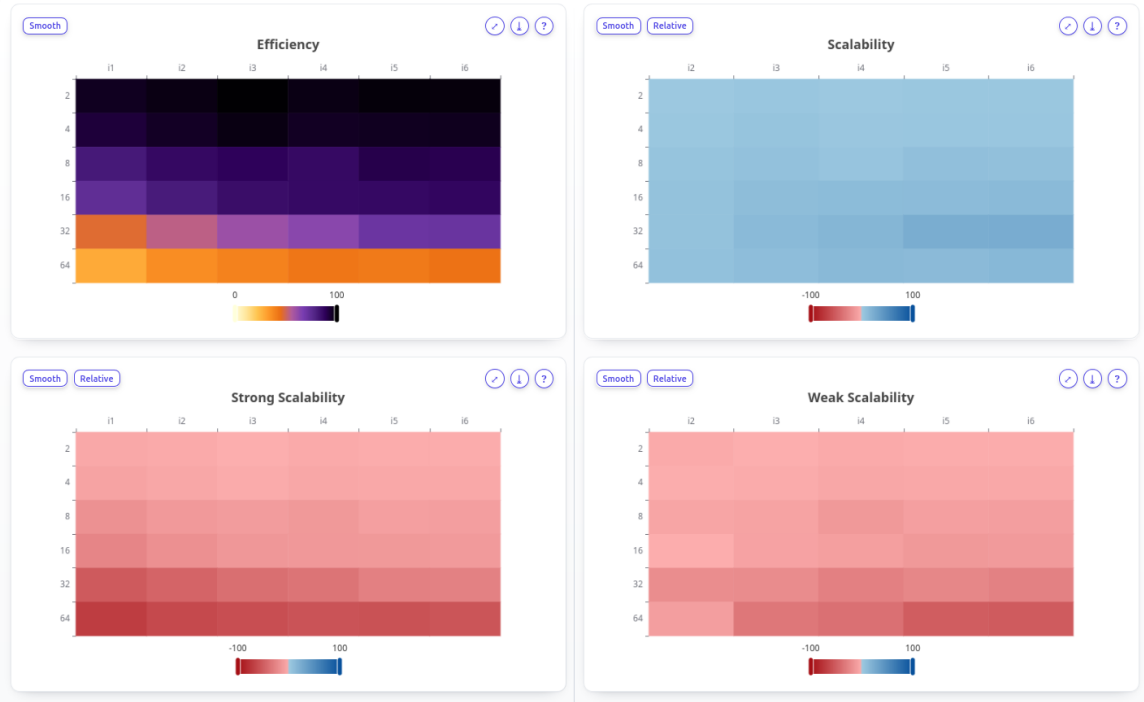


Figura 4: Versão 1, `schedule(static)`.

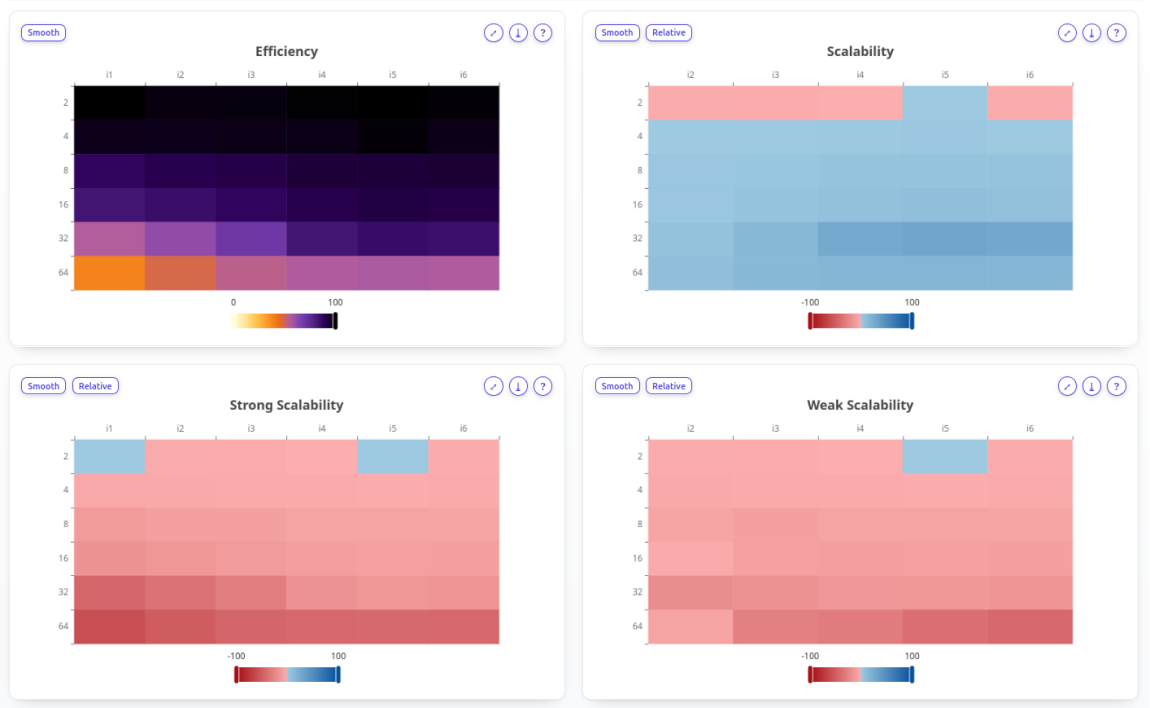


Figura 5: Versão 2, `collapse(2)` com `static`.

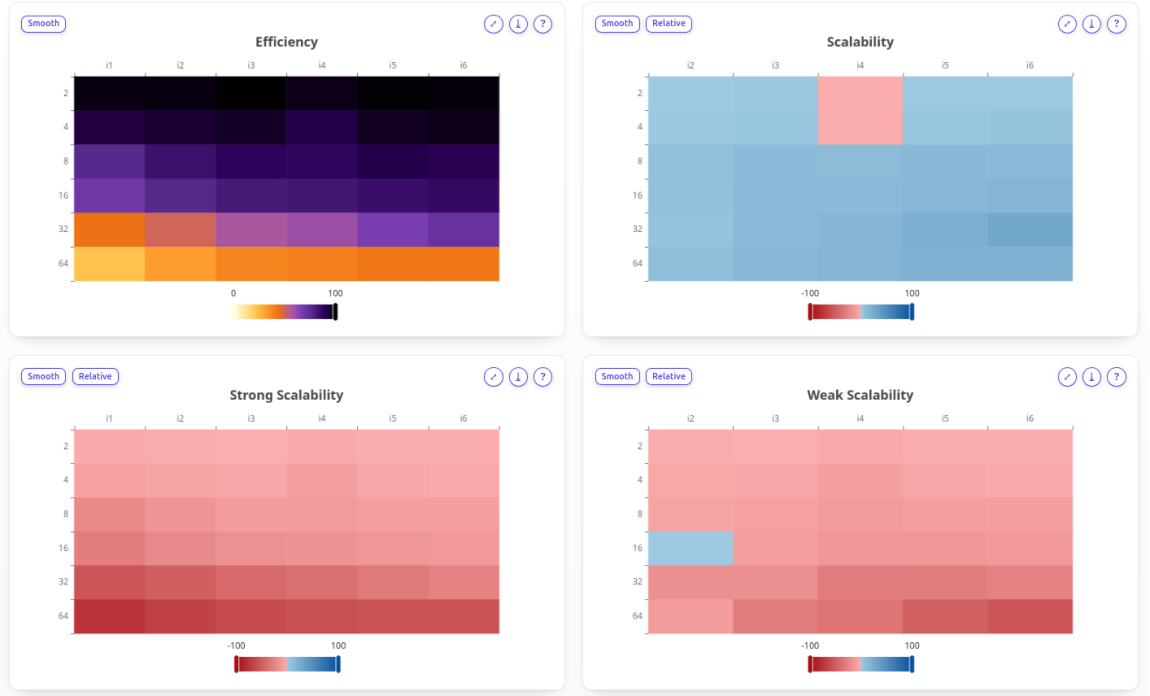


Figura 6: Versão 3, laço sem condicionais e SIMD.

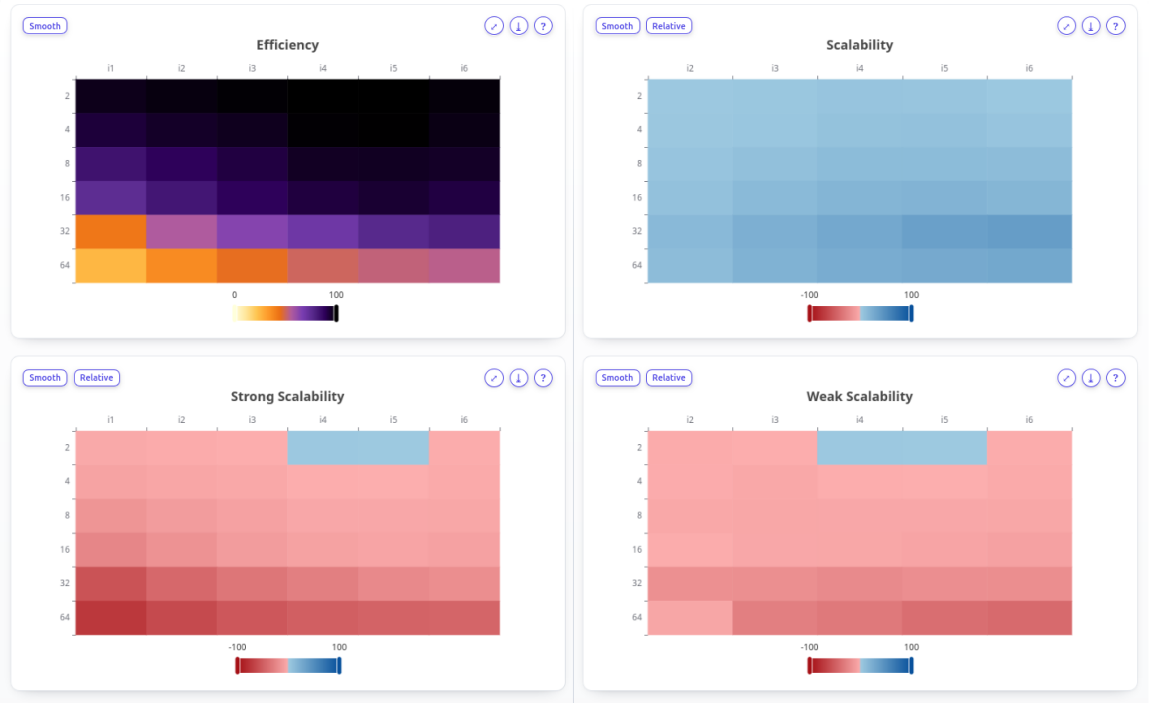


Figura 7: Versão 4, região paralela única.

4 Análise

4.1 Versão 1: o schedule é o primeiro gargalo

Gargalos. (i) O custo do escalonamento: com `dynamic` cada thread busca iterações em uma fila compartilhada, o que custa mais quanto menor o chunk, e a linha de uma thread muda de dono a cada passo, perdendo dados em cache. (ii) Só o laço `i` é paralelo: são apenas 1024 iterações, ou seja, 16 linhas por thread com 64 threads. (iii) Há uma região paralela (criação da equipe e barreira) a cada passo de tempo.

O que vemos. Com `dynamic,1` a eficiência já é menor com poucos cores (Figura 1). `dynamic,16` e `guided` melhoram com 2–4 cores, mas a eficiência despenca a partir de 16 cores (Figuras 2 e 3). O `static` (Figura 4) segura a eficiência por mais tempo, pois o custo por linha é uniforme e não há o que balancear dinamicamente.

Escalabilidade. *Forte:* ruim em todos os schedules (vermelho). *Fraca:* ruim com `dynamic` e `guided`. *Programa escalável:* com `dynamic` e `guided`, as linhas de 16 a 64 cores continuam claras até a maior entrada, ou seja, aumentar o problema não recupera a eficiência. Com `static`, a eficiência melhora para as entradas maiores.

4.2 Versão 2: `collapse(2)`

Mudança. Os laços `i` e `j` viram um só, com cerca de 10^6 iterações em vez de 1024, e o schedule passa a ser `static` fixo.

Efeito. O padrão é muito parecido com o da versão 1 `static` (Figura 5), com uma leve melhora em 32–64 cores. Mais iterações ajudam a dividir o trabalho com muitos cores, mas o gargalo principal não era o balanceamento. Continuam presentes a região paralela a cada passo e o acesso à memória, e o código ainda recalcula os vizinhos com condicionais em todo ponto.

Escalabilidade. Forte: ainda ruim, mas a queda vem mais tarde. Fraca: parcial. A eficiência melhora nas entradas maiores.

4.3 Versão 3: laço interno sem condicionais e SIMD

Mudança. O miolo do laço interno ($j = 1, \ldots, N-2$) não tem mais condicionais de contorno, e as bordas são tratadas à parte, o que permite vetorizar (`omp simd`). Também são usados ponteiros de linha em vez de `IDX()` repetido.

Efeito. Essa mudança acelera a execução com 1 thread, mas o heatmap fica parecido ou um pouco pior que o da versão 2 em 32–64 cores (Figura 6). Isso acontece porque a eficiência é medida contra o tempo de 1 thread *da própria versão*. Com a parte de cálculo mais rápida, os custos que não diminuem com p (barreira, criação da equipe, banda de memória) pesam mais na proporção. Esse é um bom exemplo de *desempenho* melhorar sem a *escalabilidade* melhorar.

Escalabilidade. Forte e fraca: semelhantes às da versão 2.

4.4 Versão 4: região paralela única

Mudança. A região `parallel` é criada uma vez, fora do laço de tempo. Dentro dela, cada passo é um `omp for` com a barreira implícita, que é necessária porque o passo $n+1$ depende do passo n. Os diagnósticos também ficam dentro da mesma região.

Efeito. Elimina o custo de criar e encerrar a equipe de threads a cada passo, o que importa quando cada thread tem pouco trabalho por passo. No heatmap (Figura 7), a versão 4 parece manter a eficiência um pouco melhor com 64 cores nas entradas maiores, com ganho moderado. A eficiência com 64 cores continua baixa. Os gargalos que sobram são a barreira a cada passo, a banda de memória (o stencil faz poucas contas por dado lido) e a malha pequena (16 linhas por thread).

Escalabilidade. Forte: continua não sendo fortemente escalável. Fraca e programa escalável: a eficiência se recupera com entradas maiores, sendo a melhor das versões nessa tendência.

4.5 Observações sobre os resultados

- Neste experimento, aumentar a entrada aumenta o número de passos, não o tamanho da malha. A melhora com entradas maiores vem de diluir custos fixos (alocação, inicialização, saída) e não de mais paralelismo por passo.
- Para avaliar escalabilidade fraca de forma mais fiel, é preciso variar o tamanho N da malha.

5 Conclusão

Nenhuma das versões é fortemente escalável: com $N = 1024$ fixo, a eficiência cai quando os cores aumentam. O que mudou entre as versões foi *onde* essa queda começa. O ganho mais claro veio de trocar `dynamic`/`guided` por `static`, já que a carga é uniforme. As versões 2, 3 e 4 trouxeram ganhos pequenos de escalabilidade; a versão 3 melhorou o desempenho sequencial, mas não a eficiência, e a versão 4 foi a que melhor manteve a eficiência em 64 cores nas entradas maiores. Em todas, aumentar o problema recupera parte da eficiência, o que indica escalabilidade parcial.

Formas de melhorar a escalabilidade.

- Usar `schedule(static)` quando o custo por iteração é uniforme.
- Reduzir o custo de sincronização: uma região paralela única e menos barreiras.
- Inicializar os dados em paralelo com a mesma partição do cálculo (*first touch*) e fixar as threads aos cores.
- Aumentar o trabalho por thread (malhas maiores) ou reduzir a parte serial e os custos fixos.

A Código-Fonte

Trechos principais de cada versão. Os fontes completos são `versao1.c` a `versao4.c`.

```c
#pragma omp parallel for schedule(runtime)
for (int i = 0; i < N; i++) {
    int im = (i == 0) ? N - 1 : i - 1;
    int ip = (i == N - 1) ? 0 : i + 1;
    for (int j = 0; j < N; j++) {
        int jm = (j == 0) ? N - 1 : j - 1;
        int jp = (j == N - 1) ? 0 : j + 1;
        double lap_u = u[IDX(ip,j,N)] + u[IDX(im,j,N)]
                     + u[IDX(i,jp,N)] + u[IDX(i,jm,N)]
                     - 4.0 * u[IDX(i,j,N)];
        /* lap_v igual, com o vetor v */
        un[IDX(i,j,N)] = u[IDX(i,j,N)] + r * lap_u;
        vn[IDX(i,j,N)] = v[IDX(i,j,N)] + r * lap_v;
    }
}
```

Código 1: Versão 1: laço externo com `schedule(runtime)`.

```c
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        /* im, ip, jm, jp calculados aqui dentro, como na versao 1 */
        ...
    }
}
```

Código 2: Versão 2: `collapse(2)` com `static`.

```c
static inline void calc_linha(const double *restrict u, const double *restrict v,
                              double *restrict un, double *restrict vn,
                              int N, int i, double r)
{
    /* im, ip e os deslocamentos de linha a, b, c calculados uma vez */
    #pragma omp simd
    for (int j = 1; j < N - 1; j++) {
        double lap_u = u[b+j] + u[a+j] + u[c+j+1] + u[c+j-1] - 4.0*u[c+j];
        double lap_v = v[b+j] + v[a+j] + v[c+j+1] + v[c+j-1] - 4.0*v[c+j];
        un[c+j] = u[c+j] + r * lap_u;
        vn[c+j] = v[c+j] + r * lap_v;
    }
    /* bordas j = 0 e j = N-1 tratadas a parte (periodico) */
}

#pragma omp parallel for schedule(static)
for (int i = 0; i < N; i++)
    calc_linha(u, v, un, vn, N, i, r);
```

Código 3: Versão 3: miolo sem condicionais e SIMD.

```c
#pragma omp parallel
{
    const double *cu = u, *cv = v;      /* ponteiros privados */
    double *nu_ = un, *nv_ = vn;

    for (int n = 0; n <= nsteps; n++) {
```

```
7         /* diagnosticos com omp single / omp for reduction */
8         if (n == nsteps) break;
9
10        #pragma omp for schedule(static)
11        for (int i = 0; i < N; i++)      /* barreira implicita ao fim */
12            calc_linha(cu, cv, nu_, nv_, N, i, r_real);
13
14        /* troca dos buffers nos ponteiros privados de cada thread */
15    }
16 }
```

Código 4: Versão 4: região paralela única (esqueleto).